

RED KFC

Ruta de Entrenamiento y Desempeño

INFORME TÉCNICO DE ARQUITECTURA

Versión 1.0 | Junio 2026 | Confidencial — Solo Ingeniería

© 2026 A2M LABS

RESUMEN EJECUTIVO

RED KFC es una Single Page Application (SPA) empresarial construida sobre un stack 100% JavaScript/TypeScript que digitaliza y centraliza la gestión del entrenamiento y desempeño del personal de restaurantes KFC Colombia. El sistema reemplaza procesos manuales en Excel con una plataforma colaborativa, auditada y disponible en tiempo real desde cualquier dispositivo con navegador web.

7

Módulos

4

Roles de Usuario

9+

Tablas DB

3

Funciones RPC

TABLA DE CONTENIDOS

1.	Visión General del Producto	2
2.	Stack Tecnológico	2
3.	Arquitectura General	3
4.	Arquitectura Frontend	4
5.	Capa de Servicios	6
6.	Base de Datos — Supabase / PostgreSQL	8
7.	Autenticación y Seguridad	11
8.	Módulos Funcionales	12
9.	Rendimiento y Escalabilidad	14
10.	Despliegue y CI/CD	15
11.	Modelo de Datos — Grupos de Evaluación	15
12.	Consideraciones de Seguridad	16
13.	Recomendaciones de Mejora	17

1. VISIÓN GENERAL DEL PRODUCTO

RED KFC (Ruta de Entrenamiento y Desempeño) es una plataforma web empresarial que digitaliza la evaluación mensual del personal de restaurantes KFC Colombia, ofreciendo visibilidad en tiempo real a nivel nacional, regional, zonal y por tienda.

Objetivos de Negocio

- %, Centralizar y automatizar la evaluación mensual por 7 grupos de competencias (AK, A, B, C, D, E, F).
- %, Proveer visibilidad en tiempo real del desempeño a nivel nacional, regional, zonal y por tienda.
- %, Gestionar la Banca de Líderes (pipeline de talento para posiciones gerenciales).
- %, Emitir y validar certificados digitales de manipulación de alimentos (Safe Hands) con QR verificable.
- %, Reemplazar procesos manuales en Excel por un sistema colaborativo y auditado.

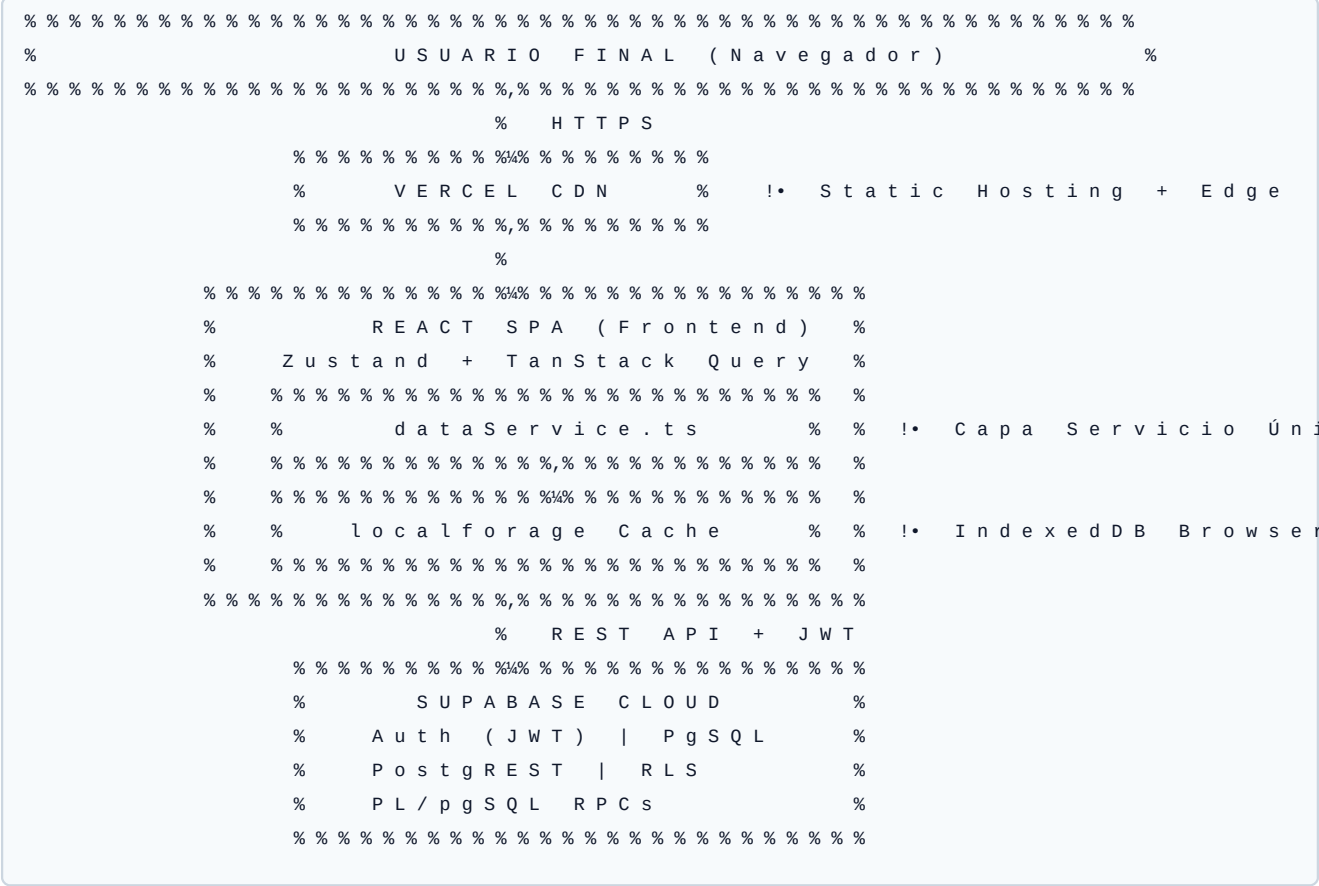
2. STACK TECNOLÓGICO

Capa	Tecnología	Versión	Rol
Framework UI	React	19.2.x	Motor de componentes SPA
Lenguaje	TypeScript	5.8.x	Tipado estático end-to-end
Build Tool	Vite	6.2.x	Bundler + Dev Server
Enrutamiento	React Router DOM	7.13.x	Navegación SPA client-side
Estado Global	Zustand	5.0.x	Store ligero y reactivo
Cache Async	TanStack React Query	5.90.x	Cache de queries + DevTools
Backend-as-a-Service	Supabase (PostgreSQL)	^2.105	DB + Auth + REST API
Persistencia Local	localforage	1.10.x	IndexedDB offline-first
Gráficas	Recharts	3.6.x	Visualización de datos
Exportación Excel	SheetJS (xlsx)	0.18.5	Generación de reportes .xlsx
Generación PDF	pdfMake + jsPDF	0.3.x/2.5.x	Certificados y reportes PDF
QR Code	qrcode	1.5.4	Generación de códigos QR
Virtualización	react-window	2.2.7	Listas virtuales (performance)
Despliegue	Vercel	—	Hosting estático + CDN global

3. ARQUITECTURA GENERAL

La aplicación sigue un patrón Offline-First SPA con 3 niveles de caché y backend serverless en Supabase Cloud. No existe un servidor de aplicaciones propio: toda la lógica reside en el cliente (React) y en funciones PostgreSQL.

Diagrama de Capas



Patrón Offline-First (3 Niveles de Caché)

Nivel	Mecanismo	TTL	Propósito
Nivel 1	Caché en Memoria (dataService._cache)	Sesión	Acceso O(1) sin I/O
Nivel 2	localforage (IndexedDB)	30 min	Persistencia local offline
Nivel 3	Supabase Cloud (PostgreSQL)	Fuente de verdad	Sincronización al inicio + polling 5 min

Flujo de Inicialización de Datos

```
% Al cargar ! initData(): Lee localforage ! Descarga nube (si caché > 30 min) ! Dete
% Al cambiar mes ! loadMonthly(): Descarga resúmenes de notas del mes selecciona
% Polling cada 5 minutos ! loadMonthly() en background si hay sesión activa.
% Al logout ! Limpia las 7 claves de la app en localforage exclusivamente.
```

4. ARQUITECTURA FRONTEND

4.1 Estructura de Módulos (Feature-Sliced Design)

Módulo	Archivo Principal	Descripción
auth/	Login.tsx	Formulario de autenticación vía Supabase Auth
dashboard/	Dashboard.tsx (32 KB)	Vista KPI con gráficas (Nacional/Regional/Zonal/Tienda)
dashboard/	GradeEditor.tsx (15 KB)	Editor inline de notas por empleado y categoría
stores/	MyStores.tsx (38 KB)	Vista detallada por tienda con evaluación y exportación
banca/	Banca.tsx (44 KB)	Gestión del pipeline de talento y certificaciones
reports/	EntriesExitsReport.tsx	Reporte de movimientos de personal
safe-hands/	SafeHands.tsx (30 KB)	Certificaciones de manipulación de alimentos
safe-hands/	PublicValidation.tsx	Validación pública de certificados QR (sin login)
admin/	AdminPanel + 5 subcomponentes	Panel CRUD: usuarios, datos, jerarquía, cierre de mes
workers/	exportWorker.ts	Web Worker para generación Excel en background

4.2 Gestión de Estado Global — Zustand Store

Estado centralizado en useAppStore con las siguientes secciones:

```
interface AppState {
  auth: { user: User | null, isAuthenticated: boolean };
  employees: Employee[];           // Empleados (filtrados por rol)
  restaurants: Restaurant[];       // Catálogo de tiendas
  filteredEmployees: Employee[];   // Empleados visibles según rol
  selectedMonth: string;           // Mes activo (YYYY-MM)
  syncStatus: 'syncing' | 'online' | 'offline';
}
```

4.3 Sistema de Roles y Control de Acceso (RBAC)

Rol	Acceso a Datos	Módulos Disponibles
ADMIN	Todos los empleados sin filtro	Dashboard, MyStores, Entries, Banca, Admin,
COORDINATOR	Empleados de sus Regiones asignadas	Dashboard, MyStores, Entries, Banca, Admin,
SPECIALIST	Empleados de sus Zonas y Tiendas asignadas	Dashboard, MyStores
LIDER	Empleados de sus Tiendas asignadas	Dashboard, MyStores, Banca, SafeHands

4.4 Enrutamiento — React Router v7

Ruta	Componente	Acceso Mínimo
/dashboard	Dashboard.tsx	Todos los roles autenticados
/my-stores	MyStores.tsx	Todos los roles autenticados
/entries-exits	EntriesExitsReport.tsx	ADMIN, COORDINATOR

/banca	Banca.tsx	ADMIN, COORDINATOR, LIDER
/admin	AdminPanel.tsx	ADMIN, COORDINATOR
/safe-hands	SafeHands.tsx	ADMIN, COORDINATOR, LIDER
/verify/:id	PublicValidation.tsx	PÚBLICO — Sin autenticación

5. CAPA DE SERVICIOS — DATASERVICE.TS

Singleton de ~1,037 líneas que actúa como intermediario entre los componentes React y Supabase/localforage. Centraliza toda la lógica de I/O, caché y transformación de datos.

5.1 Métodos de Comunicación con Supabase

- %, `supabaseFetch`: Wrapper sobre `fetch()` para GET/POST/DELETE estándar. Maneja headers JWT automáticamente.
- %, `supabaseFetchAll`: Paginación automática chunked. Hace HEAD para obtener COUNT total, luego descarga en paralelo con pool de concurrencia = 5 para evitar rate limit (HTTP 429).
- %, `supabaseFetchPaginated`: Paginación manual con offset/limit para UIs de administración.
- %, `supabaseFetchRPC`: Invoca funciones PL/pgSQL en Supabase vía el endpoint `/rest/v1/rpc/`

5.2 Estrategia de Caché

Clave de Caché	Tipo	Descripción
<code>_cache.employees</code>	<code>Employee[]</code>	Todos los empleados de la nube
<code>_cache.restaurants</code>	<code>Restaurant[]</code>	Catálogo de tiendas
<code>_cache.gradesByKey</code>	<code>Map<CECO::MES, Grades[]></code>	Notas por tienda+mes (aislamiento total)
<code>_cache.gradeIndex</code>	<code>Map<employeeId, Grades[]></code>	Índice O(1) por empleado (lazy build)
<code>_cache.summary</code>	<code>any[]</code>	Resúmenes mensuales de la vista BD
<code>_cache.hierarchy</code>	<code>HierarchyData</code>	Jerarquía Regional/Zonal/Tiendas
<code>_cache.users</code>	<code>User[]</code>	Catálogo de usuarios
<code>_cache.banca</code>	<code>BancaData</code>	Asignaciones de la banca de talento
<code>_cache.lastCloudSync</code>	<code>timestamp</code>	TTL de 30 min para datos maestros

5.3 Herencia de Notas — Lógica de Negocio Central

El sistema distingue dos tipos de grupos de evaluación:

- %, Grupos Heredables (AK, A, B, C, E): Una nota de un mes anterior vale para el mes actual si no fue re-evaluada. Se toma la nota más reciente hasta el mes consultado.
- %, Grupos No Heredables (D, F — Capacitación y SST): Solo cuentan las notas del mes exacto. No existe herencia.

```
getEffectiveGrades(employeeId, upToMonth):  
  1. Obtiene todas las notas del empleado hasta upToMonth  
  2. Filtra D y F: solo los del mes exacto (= upToMonth)  
  3. Para grupos heredables: toma la nota más reciente por grupo+categoría  
  4. Retorna el mapa consolidado de notas efectivas
```

5.4 Dual Fetch Strategy para Traslados

Para cubrir empleados que cambian de tienda, la carga usa dos fetches paralelos:

- %, Fetch A: Notas donde `restaurant_id = CECO` consultado (notas registradas en esa tienda).
- %, Fetch B: Notas donde `employee_id IN` (ids de empleados de esa tienda) — cubre notas históricas de trasladados.
- %, Deduplicación por clave `employee_id|month|group|category` antes de retornar.
- %, Rango temporal: 24 meses hacia atrás (cubre toda la herencia posible).
- %, Retry con backoff exponencial: 3 intentos × 300ms, 600ms, 1,200ms ante fallos de red.

6. BASE DE DATOS — SUPABASE / POSTGRESQL

6.1 Diagrama de Tablas

Tabla	PK	Descripción
employees	id (TEXT — Cédula)	Personal activo e inactivo con historial JSONB de movimientos
restaurants	id (TEXT — CECO)	Catálogo de tiendas: nombre, zona, región
grades	Compuesta: employee_id +	Notas de evaluación 0-100 por empleado/mes/grupo/categoría
users	id (TEXT — UUID Auth)	Perfiles de app: rol, zonas, regiones, tiendas asignadas
hierarchy	id=1 (singleton)	JSONB: meses bloqueados, config grupal, árbol
banca	id=1 (singleton)	JSONB: asignaciones de roles y certificaciones por tienda
employee_monthly_summary	Vista	Promedios por empleado/mes: avg total + avg por grupo +
monthly_group_stats	SERIAL + UNIQUE(restaurant,	Estadísticas precalculadas por tienda+mes+grupo para dashboard
safe_hands_personnel	id (TEXT — Cédula)	Personal habilitado para manipulación de alimentos
safe_hands_certs	id (UUID)	Certificados emitidos: 1 por persona, con código y fecha de
safe_hands_settings	id=1 (singleton)	Firma digital (base64) y nombre del responsable de calidad

6.2 Funciones RPC (PL/pgSQL SECURITY DEFINER)

settle_monthly_group_stats(p_month TEXT) !' INT

Calcula y persiste estadísticas en monthly_group_stats. Es IDEMPOTENTE: si el mes ya existe, lo borra y recalcula. Se ejecuta al "Asentar Notas" desde el Admin Panel.

Lógica interna en 6 CTEs:

- group_cats: Define cantidad de categorías por grupo (AK=1, A=3, B=6, C=1, D=1, E=5, F=1).
- active_emps: Empleados contratados en el mes objetivo (usando join_date / exit_date).
- emp_groups: Cross-join empleados × grupos — excluye Grupo C para empleados
- inherited/exact_month: Aplica herencia de notas (heredables vs no heredables).
- emp_group_avgs: Promedio por empleado por grupo = SUM(scores) / cat_count.
- Agrega: AVG ponderado, COUNT, approved_count ("e90%) por restaurant_id + g

backfill_monthly_group_stats() !' TEXT

Itera todos los meses distintos en grades y llama settle_monthly_group_stats() para cada uno. Ejecutar UNA SOLA VEZ para poblar el histórico de monthly_group_stats.

get_dashboard_stats(p_month, p_store_id, p_zone, p_region) !' TABLE

Función STABLE (cache-friendly en PostgreSQL) que agrega monthly_group_stats con JOIN a restaurants para filtrar por tienda/zona/región. Calcula PROMEDIO PONDERADO REAL:

```
avg = SUM(avg_score × employee_count) / SUM(employee_count)
```

6.3 Row Level Security (RLS)

Las políticas RLS de PostgreSQL controlan el acceso a nivel de fila dentro de la base de datos:

Tabla	SELECT	INSERT/UPDATE/DELETE	Condición
safe_hands_certs	Público	Todos	Sin restricción (integración inicial)
safe_hands_personnel	Público	Todos	Sin restricción (integración inicial)
safe_hands_settings	Público	Solo ADMIN	<code>auth.uid() != users.role = ADMIN</code>

& Las tablas principales (employees, grades, users) no tienen RLS explícito por fila. El control de acceso se delega al JWT + filtrado RBAC en el frontend. Se recomienda habilitar RLS completo en producción con políticas por `auth.uid()`.

6.4 Esquema de Empleado con Historial

El campo history en la tabla employees es un array JSONB que registra cada movimiento de personal:

```
history: [
  { date: "2024-03-15", restaurantName: "KFC001", action: "INGRESO" },
  { date: "2024-09-01", restaurantName: "KFC001", action: "TRASLADO" },
  { date: "2025-01-20", restaurantName: "KFC002", action: "TRASLADO" }
]
```


7. AUTENTICACIÓN Y SEGURIDAD

7.1 Flujo de Autenticación

```
Usuario ingresa username + password
    %
    %%
supabase.auth.signInWithPassword()
  Email: username@kfc.co (si no tiene @)
  JWT válido con renovación automática
    %
    %%      Si auth exitosa
supabase.from('users').select(*).ilike('username',...)
  Obtiene perfil con rol y asignaciones
    %
    %%
handleLogin(userData) != useAppStore
  initData(force=true): Descarga datos frescos
  UI habilitada según role del perfil
```

7.2 Gestión de Sesión Expirada

Listener global de supabase.auth.onAuthStateChange detecta SIGNED_OUT y ejecuta logout automático con bandera anti-recursión:

```
onAuthStateChange(event):
  if (event === 'SIGNED_OUT' && !_isLoggingOut):
    != handleLogout() limpia store + localforage
    != _isLoggingOut previene loop recursivo
```

7.3 Tabla de Controles de Seguridad

Control	Implementación	Estado
Autenticación	JWT via Supabase Auth (email+password)	Robusto
Autorización RBAC	4 roles con filtros en frontend + RLS en BD	Adecuado
Sesión expirada	Listener onAuthStateChange != logout automático implementado	
Credenciales default	Eliminadas explícitamente en migración SQL	Seguro
Variables de entorno	VITE_* en .env no commiteado (.gitignore)	Correcto
HTTPS	Forzado por Vercel en producción	Estándar
RLS Granular	Implementado en módulo Safe Hands	Parcial
Rate Limiting	Pool de 5 requests paralelos en cliente	Preventivo
Headers CSP	frame-ancestors * en vercel.json	Permisivo

8. MÓDULOS FUNCIONALES

8.1 Dashboard — Vista KPI Global

- % Selector de granularidad: Nacional! Regional! Zonal! Por Tienda.
- % KPIs por grupo: promedio, empleados evaluados, tasa de aprobación ("e90%").
- % Gráficas de barras comparativas y tendencia histórica (Recharts).
- % Nivel Zona/Regional/Nacional: usa RPC get_dashboard_stats (promedio ponderado real).
- % Nivel Tienda: usa vista employee_monthly_summary cargada localmente.

8.2 MyStores — Evaluación Detallada por Tienda

- % Lista de tiendas asignadas con resumen de KPIs.
- % Vista de tienda: descarga notas de 24 meses (Dual Fetch: por CECO + por empleados).
- % GradeEditor: editor inline con guardado en tiempo real y rollback automático si falla.
- % Filtros por estado, cargo y búsqueda por nombre/cédula.
- % Virtualización con react-window para listas de >100 empleados sin degradación.
- % Exportación a Excel via Web Worker (no bloquea el hilo principal).

8.3 Banca de Talento

- % Pipeline de líderes: asignación de roles por tienda (Gerente, Subgerente, Líder de Turno, Entrenador).
- % Gestión de certificaciones: GBR, GAR, GER, EEA por persona.
- % Ideales configurables por tienda (número objetivo de cada rol).
- % Vista de cobertura: posiciones ideales vs. cubiertas.
- % Exportación completa de la banca a Excel.

8.4 Safe Hands — Certificaciones de Manipulación de Alimentos

- % Gestión del personal habilitado para manipulación de alimentos.
- % Generación de PDF con pdfMake usando template de imagen base64 con datos superpuestos en coordenadas absolutas.
- % Generación de QR Code apuntando a /verify/{código} — URL pública verificable por cualquier persona.
- % Firma digital: el responsable de calidad sube su firma (imagen base64) que se incrusta en el PDF.
- % Validación pública en /verify/:id: sin login requerido, muestra estado del certificado (válido/expirado/no encontrado).

8.5 Panel de Administración

Componente	Función Principal
DataUploader.tsx	Importación Excel mensual: detección automática de traslados, retiros e ingresos
HierarchyViewer.tsx	Visualización y edición de la jerarquía Regional! Zonal! Tiendas
SettlementManager.tsx	Cierre de mes: bloquea el período y ejecuta settle_monthly_group_stats RPC
SstConfig.tsx	Configura títulos dinámicos de categorías D (Capacitación) y F (SST) por mes
UserManagement.tsx	CRUD completo de usuarios: crear, editar, eliminar, asignar zonas/regiones/tiendas

9. RENDIMIENTO Y ESCALABILIDAD

9.1 Optimizaciones Implementadas

- %, Web Workers: Generación de Excel en exportWorker.ts delegada a un hilo separado — no bloquea el hilo principal durante reportes de miles de filas.
- %, Virtualización de Listas: react-window renderiza solo los elementos visibles en pantalla, independientemente del tamaño del dataset.
- %, Concurrencia Controlada: Pool de máximo 5 requests paralelos en supabaseFetchAll para no exceder el rate limit de Supabase (HTTP 429).
- %, Búsqueda O(1): Uso de Map y Set en lugar de .find() O(N) para catálogos de empleados y tiendas.
- %, Normalización: TRIM(UPPER(ceco)) en frontend y en SQL elimina discrepancias por mayúsculas/espacios en CECOs.
- %, Retry con Backoff: 3 reintentos con delays 300ms/600ms/1,200ms ante fallos transitorios de red.

9.2 Escalabilidad de Base de Datos

- %, Tabla monthly_group_stats: Pre-cálculo estilo data warehouse. El RPC settle solo se ejecuta al asentar el mes, no en cada consulta del dashboard.
- %, Índices B-tree: idx_mgs_month e idx_mgs_restaurant garantizan queries O(log N) en el dashboard.
- %, Función STABLE: get_dashboard_stats marcada STABLE permite que PostgreSQL la cachee entre llamadas de la misma transacción.
- %, TTL de 30 minutos: Los datos maestros no se descargan en cada navegación, reduciendo la carga en Supabase.

10. DESPLIEGUE Y CI/CD

10.1 Plataforma de Hosting — Vercel

La aplicación se despliega como sitio estático en Vercel con CDN global. El rewrite Router maneja la navegación client-side (SPA fallback).

10.2 Build Process

```
npm run build
! ' vite build
! ' TypeScript compilation + tree-shaking
! ' Code splitting automático por ruta
! ' Output: dist/ (HTML + JS chunks + CSS + assets)

npm run dev
! ' Vite dev server en 0.0.0.0:3000
! ' HMR (Hot Module Replacement) activado
```

10.3 Variables de Entorno

- %, VITE_SUPABASE_URL: URL del proyecto Supabase (pública — va en el bundle compilado).
- %, VITE_SUPABASE_KEY: Clave anon de Supabase (pública por diseño — la seguridad real está en RLS y JWT).
- %, .env no se commitea al repositorio (.gitignore incluido). Se configura en el panel de Vercel para producción.

11. MODELO DE DATOS — GRUPOS DE EVALUACIÓN

Grupo	Nombre	Categorías	Herencia	Nota Especial
AK	All-Star Kids	1	Si	Solo aplica a empleados con >3 meses antigüedad
A	Conocimiento Producto	3	Si	—
B	Operaciones	6	Si	El grupo más complejo — 6 subcategorías
C	Cocina	1	Si	Excluido si empleado tiene <= 3 meses
D	Capacitación Mensual	1	No	Título dinámico configurable por mes (SstConfig)
E	Experiencia del Cliente	5	Si	—
F	Seguridad y Salud	1	No	Título dinámico configurable por mes (SstConfig)

!9 Criterio de Aprobación: Promedio general >= 90% en todos los grupos aplicables al empleado.

12. JERARQUÍA ORGANIZACIONAL

La estructura organizacional se almacena como JSONB en la tabla hierarchy (singleton id=1) e importable desde Excel:

```
NACIONAL
%% % REGION (ej: COSTA, BOGOTÁ, EJE CAFETERO)
    %% % ZONA / JEFE DE ÁREA (ej: ZONA NORTE)
        %% % TIENDA / RESTAURANTE (CECO, ej: KFC001)
            %% % EMPLEADOS (Cédula = PK)
```

Integración con Excel — Importación Mensual de Personal

Columnas mapeadas automáticamente desde el reporte mensual:

```
%_ Nombre_Ceco !' CECO de la tienda (normalizado a UPPERCASE)
%_ Documento !' ID del empleado (cédula)
%_ Nombre completo !' Nombre del empleado
%_ Cargo !' Título (con normalización: variaciones de "Miembro de Equipo" !' FULL o P
%_ Fecha de ingreso !' join_date (con parser de fechas seriales de Excel)
%_ Fecha fin !' exit_date + detection de retiro
```

Lógica automática de detección de movimientos:

```
%_ Traslado: Si restaurant_id cambió respecto al registro existente !' agrega TRASLA
%_ Retiro: Si aparece fecha_fin !' marca inactive + agrega RETIRO al history.
%_ Re-ingreso: Si estaba inactivo y vuelve a aparecer activo !' agrega INGRESO al hi
%_ Nuevo: Si no existe en BD !' crea registro con INGRESO como primer movimiento.
```

13. RECOMENDACIONES DE MEJORA

Corto Plazo (Inmediato)

- %, Habilitar RLS completo en tablas employees, grades y users con políticas basadas en auth.uid().
- %, Agregar logs de auditoría: tabla de eventos que registre quién modificó qué notas y cuándo.
- %, Implementar notificaciones por email (Supabase Edge Functions) al asentar el mes.

Mediano Plazo (3-6 meses)

- %, Migrar autenticación a SSO corporativo (Azure AD o Google Workspace) via Supabase OAuth providers.
- %, Implementar Supabase Realtime para sincronización en tiempo real entre múltiples usuarios simultáneos.
- %, Agregar tests unitarios (Vitest) y tests de integración (Playwright) para flujos críticos.
- %, Separar el template del certificado (~1.3 MB base64) a Supabase Storage para reducir el bundle size significativamente.

Largo Plazo (6-12 meses)

- %, Implementar Supabase Edge Functions para mover lógica de negocio crítica al servidor (validación de datos, triggers de negocio).
- %, Agregar Analytics de uso: qué funciones se usan más, tiempos de carga por región, errores en producción.
- %, Progressive Web App (PWA) con Service Worker para uso offline completo en campo sin internet.
- %, Dashboard ejecutivo adicional con análisis de tendencias históricas y alertas automáticas de rendimiento.

CONCLUSIÓN TÉCNICA

RED KFC es una aplicación empresarial moderna y funcional construida con un stack JavaScript/TypeScript de última generación. Su arquitectura offline-first, RBAC de 4 niveles, lógica de herencia de notas y módulo Safe Hands con validación QR pública representan un alto nivel de sofisticación técnica. Las principales áreas de mejora son: habilitar RLS completo, agregar cobertura de tests automatizados y reducir el bundle size del módulo Safe Hands.